SportsFolio

Cricket Athlete Funding Platform

Concise Design Document

Problem	Domestic cricketers (Ranji level) have no structured mechanism for fan-backed financial support
Core Mechanic	Player-backed tokens on a quadratic bonding curve AMM with a continuous variable pool ratio
Key Innovation	Forced coupling — every investment is also a donation. Early believers fund the success that funds their return.
Revenue	Wallet deposit fee (1%) + idle pool investment yield via licensed entity partnership
Tech Stack	Next.js | Node.js | PostgreSQL | Redis | BullMQ | Socket.io | Decimal.js

01 — Problem Statement and Solution

Ranji Trophy cricketers — the best domestic players in India outside the IPL — routinely cannot afford equipment, travel, or coaching. When they need money, they call family and friends. That money is given informally, with no return even if the player later becomes a star. SportsFolio formalises this existing behaviour into a structured financial product.

The Two User Personas — Core Design Anchor

Persona	Who They Are	Motivation	What They Get
Believer (early)	Friends, family, coaches, local fans	Genuine belief in the person. Was going to give money anyway.	Token at near-zero price. Massive upside if athlete succeeds. Proof of early belief.
Profit Seeker (late)	Market participants, sports investors	Sees price history, pool depth, athlete performance data.	Asymmetric return from curve position. Small donation component is acceptable cost.

Why This Solves the Cold Start Problem

The cold start problem — why does anyone invest first — is answered by the Believer persona. They exist before the platform does. The platform does not manufacture early demand. It channels demand that already exists in personal relationships. The Believer's investment funds the athlete's performance, which attracts the Profit Seeker, whose investment funds the Believer's return. This chain is the entire product.

02 — Financial Mechanics

2.1 Bonding Curve AMM

A bonding curve sets token price as a deterministic function of current supply. No order book. No counterparty. The curve always buys and always sells at a price derived purely from how many tokens currently exist.

```
Price(S) = k x S^2 (quadratic bonding curve)

Buy cost (S1 -> S2) = k x (S2^3 - S1^3) / 3 [integral of k.s^2 ds]
Sell proceeds (S2->S1) = k x (S2^3 - S1^3) / 3 [same formula, reversed]

Pool balance invariant: Pool = k x S^3 / 3 (always, by construction)
=> If all tokens are sold, pool pays exactly what all investors paid in. Always solvent.

Tokens are fully fractional. Any INR amount buys the mathematically exact token quantity.
```

Why quadratic? Linear curves give modest early-investor advantage. Cubic curves are too aggressive and make early tokens nearly worthless at scale. Quadratic rewards early belief meaningfully without making late-stage participation inaccessible.

2.2 Variable Pool Ratio — The Core Innovation

Every purchase is split between the liquidity pool (investor-backed) and a donation to the athlete. The split is not fixed — it shifts continuously as the token price grows. Early stage: athlete needs maximum funding, token is cheap, pool is thin but that is fine because early investors get enormous token quantities per rupee. Late stage: market needs deep liquidity, donation component is minimal.

```
pool_ratio(P) = 0.10 + 0.89 x (P / (P + P_mid))

At P = 0: pool = 10%, donation = 90% (Believer phase)
At P = P_mid: pool = 54.5%, donation = 45.5% (transition)
As P -> inf: pool -> 99%, donation -> 1% (Market phase)

P_mid configured per athlete at listing (low for unknown players, high for established stars).

Per-transaction (numerical integration, 1000 steps, all Decimal.js arithmetic):
pool_deposit = SUM[ pool_ratio(price_at_step) x cost_at_step ]
donation = SUM[ (1 - pool_ratio(price_at_step)) x cost_at_step ]
```

Why continuous and not discrete thresholds? Discrete thresholds create cliff effects — a single trade crossing a boundary triggers a sudden ratio jump that can be gamed. A smooth function eliminates this entirely. There is no exploitable boundary.

2.3 Early Investor Advantage — The Honest Math

The early investor's pool deposit is small (10% of payment at early stage) but they receive a disproportionately large number of tokens because the curve price is near zero. This is the curve position advantage — and it completely dominates the pool ratio disadvantage.

	Early Investor	Late Investor
Total paid	Rs 1,000	Rs 1,000
Pool deposit	Rs 100 (10%)	Rs 900 (90%)
Donation sent	Rs 900	Rs 100
Tokens received	31.07	1.38
Exit value (at supply=512)	Rs 76,000+	Rs 3,500

Where Does the Money Come From?

The bonding curve does not create money. It redistributes it. The early investor's profit comes directly from later investors' pool deposits. This is true of every asset market. What makes SportsFolio different: the donation component means real value was created — an athlete was funded — not just money circulated between speculators. The mechanics are fully disclosed to every investor.

03 — Revenue Model

Revenue Stream	Mechanism	Legal Status	Scale Needed
Wallet deposit fee	1% charged when user tops up wallet. Rs 990 credited for Rs 1,000 deposited.	Clearly legal. Platform's money once collected.	Viable from day one
Idle pool investment	80% of aggregate pool deployed into T-bills / liquid MFs. 20% reserve kept liquid.	Requires RBI PPI licence or licensed NBFC partner.	Requires Rs 2-3 crore pool aggregate
Athlete listing fee	One-time fee for verified athlete onboarding and profile creation.	Clearly legal.	Viable from day one
Withdrawal fee	Small flat fee on bank transfer from wallet.	Clearly legal.	Viable from day one

Fee at deposit, not withdrawal — charging at deposit means the user always knows exactly what they have. Charging at withdrawal creates a moment of perceived betrayal that erodes trust. Major platforms charge at withdrawal to reduce deposit friction, but they sacrifice honesty to do it. SportsFolio's brand is built on transparency. Fee at deposit is consistent with that brand.

24-hour withdrawal queue — sell proceeds are instantly credited to the SportsFolio wallet. Bank withdrawal is processed within 24 hours. This gives the platform a daily settlement window to aggregate all pending withdrawals, compare against liquid reserve, and liquidate just enough deployed capital if needed. In practice with a 20% reserve, liquidation is rarely required.

04 — Financial System Integrity

4.1 Floating Point — Never Use It for Money

Native floating point cannot represent most decimals exactly. 0.1 + 0.2 = 0.30000000000000004 in JavaScript. This error compounds across thousands of trades and eventually breaks reconciliation. Every monetary and token calculation in SportsFolio uses **Decimal.js** with 28-digit precision. All monetary database columns are **DECIMAL(20,8)** — never FLOAT or DOUBLE. All amounts cross API boundaries as **strings**, never JSON numbers.

4.2 Ledger System

SportsFolio never updates balances directly. Every financial event creates an append-only ledger entry. Current balance is always derived by summing the ledger — never stored as a mutable field. The complete financial history of every account is always reconstructable.

```
Every buy writes 4 ledger entries:
DEBIT investor_wallet -amount_total
CREDIT athlete_pool +pool_deposit
CREDIT athlete_donation_queue +donation_amount
CREDIT tokens_minted +token_quantity

Nightly verification: pool_balance = k x S^3 / 3 (any mismatch -> alert)
```

4.3 Idempotency Keys

Client generates a UUID before every trade request and sends it with the request. Server checks this key before processing — if it has seen it before, it returns the cached result without re-executing. Network retries are always safe. Double-charging is impossible. Keys are stored with a UNIQUE constraint and expire after 24 hours.

4.4 Race Conditions

Two simultaneous trades for the same athlete's token would both read the same supply, both calculate a price, and both attempt to write — producing an incorrect pool balance. Two mechanisms prevent this:

- **Per-athlete trade queue (primary)** — all buy/sell requests for athlete A route to a dedicated BullMQ queue with a single consumer. Trades are serialised. Zero concurrent writes possible.
- **SELECT FOR UPDATE (secondary)** — inside the database transaction, the token row is locked before any calculation. No other transaction can read or write it until the current one commits.
- **Optimistic locking (tertiary guard)** — a version column on the tokens table increments on every update. Any update that does not match the expected version fails and retries from scratch.

4.5 Daily Reconciliation

A nightly cron job verifies every financial invariant: pool balance equals curve integral, investor wallet balances match ledger sums, platform revenue matches fee collections. Any discrepancy raises an alert and routes to manual review. Nothing is corrected automatically.

05 — Fault Tolerance

Mechanism	What It Protects Against	How
DB Transaction Atomicity	Partial trade state (tokens minted but pool not updated)	Every trade is one ACID transaction. Any failure rolls back entirely.
Saga Pattern	External payment API failure mid-flow	Write PENDING donation record first. Execute payment. Mark COMPLETED. Failed payments retry, then compensate.
Exponential Backoff Retry	Transient network failures to payment gateway or external APIs	Retry after 1s, 2s, 4s. Idempotency key passed on every retry. Safe to retry always.
Dead Letter Queue	Jobs that fail after all retries silently disappearing	All exhausted-retry jobs route to DLQ. Admin notification raised. Manual review required.
Health Checks + PM2	Crashed services going undetected	Every service exposes /health. PM2 monitors and auto-restarts crashed processes.
Bot Runner Isolation	Bot activity degrading real user API response times	Bot Runner is a completely separate process. If it crashes, the main API is unaffected.

06 — Bot Trading System

Why Bots Exist

At launch, SportsFolio will have few real users. Without activity, every athlete's price chart is a flat horizontal line. A flat chart communicates nothing, creates no engagement, and gives investors nothing to evaluate. System bots are an explicit, declared substitute for real users. They produce organic-looking price action that makes the platform functional from day one. Bot activity is labelled as such in the UI — it is not hidden.

System Bots — Three Built-in Strategies

Bot	Strategy	Logic	Market Role
Momentum Bot	Trend following	Buy on consistent uptrend. Sell on reversal.	Amplifies genuine price moves
Mean Reversion Bot	Statistical	Buy when price below rolling avg. Sell above avg.	Anchors price near fair value
Noise Trader Bot	Random	Random buy/sell with random sizing on fixed interval.	Simulates organic market chaos

User Config Bots

Investors configure their own bots via a parameter form — no coding required. Configurable parameters: trigger type (price change % / MA crossover / time interval), direction (buy/sell/both), threshold %, trade size (fixed INR or % of wallet), cooldown period, stop loss %, take profit %. All bots compete on a live leaderboard ranked by ROI.

Scalable Bot Runner Architecture

A single process loop does not scale. The Bot Runner uses a distributed worker pool: each bot tick is a BullMQ job, consumed by N parallel workers. Bot state (rolling averages, price window, wallet balance, position) is stored in Redis — not in memory. Worker restarts resume from exact saved state. Scaling the bot system means starting more workers. No code changes required.

07 — Design Patterns and Principles

Design Patterns

Pattern	Type	Where Applied	Why
Strategy	Behavioural	Bot trading strategies (Momentum, MeanReversion, Noise, UserConfig)	New strategies added by implementing one interface. BotRunner never changes.
Observer	Behavioural	MarketEventBus -> WebSocketServer, LeaderboardService, PriceHistoryLogger	TradeService fully decoupled from all real-time consumers. Zero knowledge of subscribers.
Repository	Structural	AthleteRepo, TokenRepo, TradeRepo, BotRepo	Business logic never writes SQL. All services testable with mock repositories.
Factory	Creational	BotFactory.create(type, config) -> concrete Bot subclass	BotRunner never directly instantiates bots. New bot type: extend factory only.
Saga	Distributed	Donation transfer to athlete bank account	Handles failure of external payment API without corrupting core trade state.

SOLID Principles

Principle	Applied As
Single Responsibility	BondingCurveEngine: pricing only. TradeService: orchestration only. KycService: verification only. BotRunner: scheduling only.
Open / Closed	Strategy interface: new bot types added without touching BotRunner. Token base class: new token types without touching engine.
Liskov Substitution	MomentumBot, MeanReversionBot, NoiseBot, UserBot all substitutable for abstract Bot. BotRunner works with any.
Interface Segregation	Strategy declares only tick(). PaymentGateway separate from NotificationGateway. TokenRepository separate from TradeRepository.
Dependency Inversion	TradeService depends on BondingCurveEngine interface. BotRunner depends on Strategy interface. All services depend on Repository interfaces.

OOP Pillars

Pillar	Applied As
Encapsulation	BondingCurveEngine.poolBalance is private. Athlete bank details never serialised to API responses. Bot wallet only modified via executeTrade().

Pillar	Applied As
Inheritance	Abstract Token -> AthleteToken. Abstract Bot -> MomentumBot, MeanReversionBot, NoiseBot, UserBot. All inherit executeTrade() with no duplication.
Polymorphism	BotRunner.runTick(bots: Bot[]) calls tick() polymorphically. Each concrete subclass returns TradeSignal using its own logic. Runner never checks type.
Abstraction	Strategy interface: tick(MarketState) -> TradeSignal. PaymentGateway interface. TokenRepository interface. All hide implementation details from consumers.

08 — Architecture and Scaling Strategies

Service Overview

Service	Technology	Responsibility
Frontend	Next.js + Tailwind	All UI — profiles, buy/sell, dashboard, leaderboard
Backend API	Node.js + Express	REST + WebSocket, auth, trade orchestration, idempotency
Database	PostgreSQL + Prisma	All entities, ledger, audit log, idempotency keys
Bonding Curve Engine	Node.js module	Pure Decimal.js pricing math — stateless, independently testable
Bot Runner	Isolated Node.js worker	Bot scheduler and worker pool. Separate process from API.
Cache	Redis	Token prices, session tokens, rate limits, bot state, leaderboard
Queue	BullMQ (Redis-backed)	Per-athlete trade queues, bot tick jobs, withdrawal processing
Real-time	Socket.io	Live price chart, leaderboard refresh, market activity feed

Scaling Strategies — In Order of When Needed

Layer	When to Add	What It Enables
Redis Cache	Day 1	Token prices served from memory. No DB hit per price read. Leaderboard as sorted set.
Per-Athlete Queue	Day 1	Serialises trades. Solves race conditions. Enables stateless horizontal API scaling.
PostgreSQL Read Replica	10k daily users	Read queries (portfolios, history) route to replica. Writes to primary only.
Horizontal API Scale	10k daily users	Multiple API instances behind load balancer. Stateless (JWT + Redis sessions).
Kafka Event Streaming	100k daily users	Trade events streamed to analytics, notifications, history in parallel. Replay on failure.
DB Sharding	1M daily users	Shard by athlete_id via Citus. Each shard independent. Linear throughput scaling.

09 — Market Guards and Anti-Manipulation

Guard	Trigger	Response
Price circuit breaker	Price moves > 5% in any 5-minute window	Pause trading on that athlete's token for 15 minutes
Volume circuit breaker	Trade volume > 3x 7-day average in 1 hour	Pause and notify admin. Manual review to resume.
Pool drain guard	Single sell would reduce pool by > 15%	Reject trade. Return max allowed sell amount to user.
Max trade size	Single trade > 10% of current pool size	Reject. Return the maximum permitted amount.
Self-investment block	Athlete's KYC bank account buys own token	Hard block at trade service layer.
Wash trade detection	Same account buys and sells same token within 10 minutes repeatedly	Flag for review. Temporary rate limit applied.
Supply concentration	Single account holds > 20% of any token supply	Block further buys until concentration falls below threshold.
Bot wallet cooldown	User bot wallet topped up within 24hrs of stop-loss firing	24-hour cooldown enforced. Prevents rapid re-deployment.

System bots are excluded from circuit breaker triggers — they are market makers, not manipulators. User bots are subject to all circuit breaker rules plus the additional bot-specific guards above.

10 — Entity Relationship Diagram

Nine entities. Blue field names are Primary Keys. Purple field names are Foreign Keys. LEDGER and AUDIT_LOG are append-only — never updated, only inserted. TRADES enforces idempotency_key as a UNIQUE constraint. All monetary fields are DECIMAL(20,8).

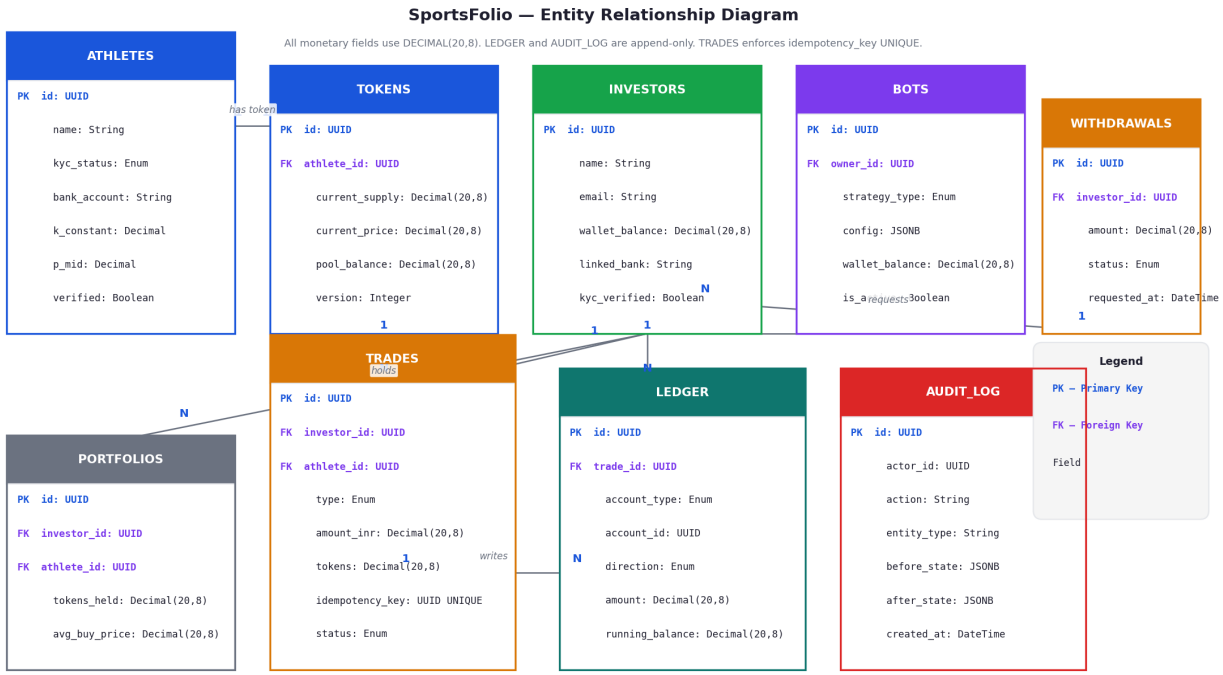

Figure 1 — SportsFolio Entity Relationship Diagram

Key Design Decisions in the Schema

Decision	Reason
tokens.version column	Optimistic locking — secondary guard against race conditions alongside SELECT FOR UPDATE
trades.idempotency_key UNIQUE	Database-level guarantee that no two trades share a key — catches application-layer bugs
ledger is append-only	Complete audit history. Any account state reconstructable at any point in time.
audit_log is append-only	Immutable record of every admin action, system event, and state change across the platform
portfolios as explicit table	Avoids expensive JOIN + SUM query on trades for every portfolio read. Denormalised for read performance.
bots.owner_id nullable	NULL owner_id = system bot. Non-null = user bot. Single table for both types.

Decision	Reason
withdrawals as separate table	Decouples the 24-hour withdrawal queue from the real-time wallet balance. Processed independently in batch.